

Bloque 6 — Mini-especificaciones funcionales y técnicas

Rendiciones: R003, R004, R002, R001 y R005–R009

1. Propósito del bloque

Este bloque construye la base necesaria para que una persona pueda:

- Estar habilitada como responsable de rendición.
- Crear una rendición en borrador.
- Registrar el período y motivo.
- Asociar uno o varios fondos entregados.
- Registrar facturas, boletas, peajes y otros gastos.
- Incorporar múltiples comprobantes.
- Detectar documentos duplicados o inconsistentes.
- Identificar gastos sin respaldo.
- Calcular fondos, gastos, montos aprobables y diferencias.
- Consultar rendiciones mediante filtros.

Todavía no se implementa el flujo formal de:

- Presentación.
- Revisión.
- Observación.
- Aprobación.
- Rechazo.
- Liquidación.
- Devolución.
- Reembolso.
- Cierre.

Esas operaciones se desarrollarán en R010–R017.

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
R003	Responsables de rendición	4 h	Backend	Parcial	Personas habilitadas y datos históricos
R004	Categorías y subcategorías	4 h	Backend	Alta	Catálogo administrable

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
R002	Crear y editar rendición	6 h	Backend	Alta, con ajustes	Cabecera en borrador
R001	Lista y búsqueda	4 h	Frontend	Parcial	Consulta filtrable y paginada
R005	Entregas de fondo	6 h	Backend	Parcial	Múltiples fondos y saldo acumulado
R006	Detalle de gastos	8 h	Backend	Alta, con ajustes	Registro estructurado de gastos
R007	Adjuntos y previsualización	8 h	Frontend	Parcial	Múltiples archivos protegidos
R008	Validación documental	10 h	Backend	Parcial	Controles automáticos y observaciones
R009	Cálculos de rendición	4 h	Backend	Parcial	Totales y diferencias consistentes

El bloque suma **54 horas**.

La Carta Gantt exige, entre otros resultados:

- Filtros por responsable, estado, fecha y número.
- Borradores editables con validaciones de cabecera.
- Responsables seleccionados desde personal y conservación histórica.
- Categorías activas y sin duplicados.
- Múltiples entregas de fondos.
- Registro de facturas, boletas, peajes y otros documentos.
- Subida, previsualización, descarga y reemplazo controlado.
- Control de duplicados, fechas, montos, proveedor y ausencia de respaldo.
- Totales, fondos y diferencias consistentes.

3. Orden lógico de implementación

Aunque la numeración comienza con R001, la secuencia técnica recomendada es:

```

R003 Responsables
→ R004 Categorías y subcategorías
→ R002 Crear y editar rendición
→ R001 Lista y búsqueda
→ R005 Entregas de fondo
→ R006 Detalle de gastos
→ R007 Adjuntos

```

- R008 Validación documental
- R009 Cálculos

La razón es que una rendición necesita previamente:

- Un responsable habilitado.
- Catálogos de gastos.
- Reglas de numeración.
- Estados iniciales.
- Permisos.

La lista R001 se construye después del alta R002 porque necesita datos reales y cálculos anotados para probar filtros, estados y totales.

4. Flujo funcional del bloque

```
Responsable habilitado
→ categorías configuradas
→ rendición creada en borrador
→ fondos vinculados
→ gastos registrados
→ documentos adjuntados
→ validaciones ejecutadas
→ totales calculados
→ rendición preparada para presentación
```

5. Cobertura actual de modelos

El archivo de Administración ya contiene:

```
ResponsableRendicion
CategoriaGasto
SubcategoriaGasto
Rendicion
EntregaFondo
DetalleRendicion
AprobacionRendicion
LiquidacionRendicion
```

`ResponsableRendicion` puede relacionarse uno a uno con un usuario y conserva nombre, RUT, cargo, estado y observaciones.

Las categorías y subcategorías ya permiten activar o desactivar registros y evitan repetir el mismo nombre de subcategoría dentro de una categoría.

`Rendicion` contiene número, responsable, período, motivo, lugar, estado y fechas del proceso. También calcula fondos, monto rendido y monto aprobado.

`EntregaFondo` admite múltiples entregas por rendición, diferentes formas de entrega, referencias y anulación.

`DetalleRendicion` ya registra documento, proveedor, categoría, forma de pago, montos, revisión, observaciones y un comprobante. Además, impide aprobar más que el total y exige justificación para gastos sin documento.

6. Ajustes transversales requeridos

6.1 Conservación histórica del responsable

Actualmente una rendición apunta a `ResponsableRendicion`.

Si posteriormente se modifica:

- Nombre.
- RUT.
- Cargo.

las rendiciones antiguas mostrarían los nuevos datos.

Se recomienda agregar a `Rendicion`:

```
responsable_nombre = models.CharField(  
    max_length=150,  
    blank=True,  
)  
responsable_rut = models.CharField(  
    max_length=20,  
    blank=True,  
)  
responsable_cargo = models.CharField(  
    max_length=100,  
    blank=True,  
)
```

Estos campos se completarán al crear la rendición:

```
Responsable actual  
→ copiar nombre, RUT y cargo  
→ conservar snapshot
```

6.2 Numeración automática

El campo `Rendicion.numero` ya es único, pero falta definir cómo se genera.

Formato recomendado:

REN-2026-000001

La numeración debe generarse dentro de una transacción para evitar duplicados.

Podrá utilizarse:

```
class SecuenciaDocumento(models.Model):
    tipo = models.CharField(max_length=30, unique=True)
    ultimo_numero = models.PositiveIntegerField(default=0)
```

O una secuencia específica para rendiciones.

6.3 No eliminación física después de presentación

Durante el borrador puede permitirse eliminar un detalle recién creado, según la política final.

Después de presentar:

- Los gastos no se eliminan físicamente.
- Las entregas no se eliminan.
- Los adjuntos no se sobrescriben.
- Las correcciones deben quedar registradas.

Se recomienda agregar a `DetalleRendicion`:

```
anulado = models.BooleanField(default=False)
anulado_en = models.DateTimeField(null=True, blank=True)
anulado_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="detalles_rendicion_anulados",
)
motivo_anulacion = models.TextField(blank=True)
```

6.4 Separación entre validación automática y revisión humana

El campo:

```
DetalleRendicion.estado_revision
```

representa la decisión humana:

- Pendiente.
- Observado.
- Aprobado.
- Rechazado.

R008 requiere controles automáticos anteriores a esa revisión.

Por ello no se debe utilizar `estado_revision` para representar:

- Documento duplicado.
- Fecha fuera del período.
- RUT inválido.
- Falta de comprobante.
- Diferencia de montos.

Se necesita una validación documental separada.

R003 — Responsables de rendición

1. Objetivo

Administrar las personas autorizadas para:

- Recibir fondos.
- Registrar gastos.
- Crear rendiciones.
- Corregir observaciones.
- Recibir reembolsos.
- Realizar devoluciones.

La tarea debe permitir seleccionar personal existente y conservar la información histórica utilizada en cada rendición.

2. Modelo principal

```
ResponsableRendicion
```

Campos actuales:

- Usuario.
- Nombre.
- RUT.
- Cargo.
- Activo.
- Observaciones.

La relación con `User` es opcional, lo que permite registrar responsables sin acceso directo al sistema.

3. Tipos de responsable

Responsable con acceso al sistema

```
ResponsableRendicion.user != None
```

Puede:

- Ver sus rendiciones.
- Crear borradores.
- Adjuntar comprobantes.
- Presentar.
- Corregir observaciones.

Responsable sin usuario

```
ResponsableRendicion.user = None
```

Sus rendiciones serán registradas por:

- Finanzas.
 - Administración.
 - Un delegado autorizado.
-

4. Ajustes recomendados

Agregar:

```
correo = models.EmailField(blank=True)
telefono = models.CharField(max_length=40, blank=True)
area = models.CharField(max_length=100, blank=True)
```

Opcionalmente:

```
supervisor = models.ForeignKey(  
    "self",  
    on_delete=models.SET_NULL,  
    null=True,  
    blank=True,  
    related_name="responsables_supervisados",  
)
```

El supervisor solo debe agregarse si forma parte real del flujo de aprobación.

5. Reglas de negocio

1. Un usuario solo puede tener un ResponsableRendicion.
 2. El nombre es obligatorio.
 3. El RUT debe normalizarse.
 4. No pueden existir dos responsables activos con el mismo RUT.
 5. Un responsable inactivo no puede utilizarse en nuevas rendiciones.
 6. Un responsable inactivo permanece visible en el historial.
 7. Desactivar no elimina rendiciones.
 8. No se debe modificar el responsable de una rendición presentada sin una acción administrativa especial.
 9. La rendición conserva snapshot de nombre, RUT y cargo.
 10. Vincular un usuario no debe otorgar automáticamente permisos de aprobación.
-

6. URLs

```
/administracion/rendiciones/responsables/  
/administracion/rendiciones/responsables/nuevo/  
/administracion/rendiciones/responsables/<pk>/  
/administracion/rendiciones/responsables/<pk>/editar/  
/administracion/rendiciones/responsables/<pk>/activar/  
/administracion/rendiciones/responsables/<pk>/desactivar/
```

7. Vistas

```
ResponsableRendicionListView  
ResponsableRendicionDetailView  
ResponsableRendicionCreateView  
ResponsableRendicionUpdateView  
ResponsableRendicionActivarView  
ResponsableRendicionDesactivarView
```

8. Formulario

Campos:

- Usuario.
- Nombre.
- RUT.
- Cargo.
- Área.
- Correo.
- Teléfono.
- Activo.
- Observaciones.

Al seleccionar un usuario se pueden completar inicialmente:

- Nombre.
- Correo.

Estos datos seguirán siendo editables como información administrativa.

9. Template de detalle

Mostrar:

Identificación

- Nombre.
- RUT.
- Cargo.
- Área.
- Usuario relacionado.
- Estado.

Resumen

- Rendiciones en borrador.
- Rendiciones pendientes.
- Fondos entregados.
- Fondos pendientes de regularización.
- Última rendición.

Los indicadores se completarán progresivamente.

10. Permisos

```
view_responsablerendicion  
add_responsablerendicion
```

```
change_responsablerendicion  
desactivar_responsablerendicion
```

El responsable común no debe administrar el catálogo de responsables.

11. Auditoría

Registrar:

- Alta.
 - Vinculación con usuario.
 - Cambio de RUT.
 - Cambio de cargo.
 - Activación.
 - Desactivación.
-

12. Pruebas

- Responsable con usuario.
 - Responsable sin usuario.
 - Usuario duplicado.
 - RUT duplicado.
 - RUT normalizado.
 - Desactivar responsable.
 - Impedir uso de responsable inactivo.
 - Conservar rendiciones históricas.
 - Conservar snapshot.
 - Permisos.
-

13. Criterios de aceptación

- Se pueden registrar responsables.
 - Se pueden vincular con usuarios.
 - Un responsable puede existir sin cuenta.
 - No existen duplicados funcionales.
 - Los inactivos no aparecen en nuevos formularios.
 - Las rendiciones conservan datos históricos.
 - La actividad queda auditada.
-

R004 — Categorías y subcategorías de gasto

1. Objetivo

Mantener un catálogo jerárquico para clasificar los gastos rendidos.

Ejemplos:

```
Movilización
├─ Combustible
├─ Peaje
├─ Estacionamiento
└─ Transporte público

Alimentación
├─ Desayuno
├─ Almuerzo
└─ Cena

Materiales
├─ Ferretería
├─ Insumos técnicos
└─ Elementos de seguridad
```

2. Modelos involucrados

```
CategoriaGasto
SubcategoriaGasto
```

Los modelos ya tienen estado activo y unicidad de subcategoría dentro de cada categoría.

3. Campos recomendados para categoría

Agregar:

```
codigo = models.CharField(
    max_length=30,
    blank=True,
)
descripcion = models.TextField(blank=True)
requiere_documento = models.BooleanField(default=True)
```

```
permite_sin_documento = models.BooleanField(default=False)
requiere_proveedor = models.BooleanField(default=True)
```

Opcionalmente:

```
monto_maximo_sin_documento = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    null=True,
    blank=True,
)
```

Este límite debe configurarse únicamente si existe una política administrativa real.

4. Campos recomendados para subcategoría

```
codigo = models.CharField(
    max_length=30,
    blank=True,
)
descripcion = models.TextField(blank=True)
```

La subcategoría podrá heredar las reglas de la categoría.

En una fase posterior podría permitir reglas propias.

5. Reglas de negocio

1. El nombre de categoría es obligatorio.
2. No pueden existir categorías duplicadas por diferencias de mayúsculas.
3. Una subcategoría pertenece a una sola categoría.
4. No puede repetirse una subcategoría dentro de la misma categoría.
5. Una categoría inactiva no aparece en gastos nuevos.
6. Al desactivar una categoría se deben desactivar o bloquear sus subcategorías.
7. Los gastos históricos conservan la categoría.
8. No se eliminan categorías utilizadas.
9. Una subcategoría seleccionada debe pertenecer a la categoría seleccionada.
10. Las categorías pueden definir requisitos documentales.

6. URLs

```
/administracion/rendiciones/categorias/
/administracion/rendiciones/categorias/nueva/
```

```
/administracion/rendiciones/categorias/<pk>/editar/  
/administracion/rendiciones/categorias/<pk>/desactivar/  
/administracion/rendiciones/categorias/<pk>/subcategorias/  
/administracion/rendiciones/subcategorias/nueva/  
/administracion/rendiciones/subcategorias/<pk>/editar/
```

7. Vistas

```
CategoriaGastoListView  
CategoriaGastoCreateView  
CategoriaGastoUpdateView  
CategoriaGastoDesactivarView  
SubcategoriaGastoCreateView  
SubcategoriaGastoUpdateView  
SubcategoriaGastoDesactivarView
```

8. Interfaz

La pantalla principal podrá utilizar una tabla jerárquica:

Categoría	Subcategorías	Requiere documento	Estado	Acciones
-----------	---------------	--------------------	--------	----------

Al expandir una categoría se mostrarán sus subcategorías.

9. Endpoint dependiente

Para el formulario de gastos:

```
/administracion/rendiciones/subcategorias/?categoria=<pk>
```

Solo devolverá subcategorías activas de la categoría seleccionada.

10. Permisos

```
view_categoriagasto  
add_categoriagasto  
change_categoriagasto  
desactivar_categoriagasto  
  
view_subcategoriagasto
```

```
add_subcategoriagasto  
change_subcategoriagasto  
desactivar_subcategoriagasto
```

11. Auditoría

Registrar:

- Alta.
 - Cambio de nombre.
 - Cambio de requisitos.
 - Activación.
 - Desactivación.
 - Cambio de categoría de una subcategoría.
-

12. Pruebas

- Categoría válida.
 - Categoría duplicada.
 - Subcategoría válida.
 - Duplicado dentro de categoría.
 - Mismo nombre en otra categoría.
 - Categoría inactiva.
 - Subcategoría de categoría incorrecta.
 - Uso histórico.
 - Filtro dependiente.
 - Permisos.
-

13. Criterios de aceptación

- El catálogo es administrable.
 - No existen duplicados funcionales.
 - Se pueden activar y desactivar elementos.
 - Los gastos históricos permanecen consistentes.
 - Los formularios solo muestran opciones activas.
 - Categoría y subcategoría siempre coinciden.
-

R002 — Crear y editar rendición

1. Objetivo

Permitir crear una rendición en estado borrador y modificar su cabecera mientras las reglas del flujo lo permitan.

2. Modelo principal

Rendicion

Campos existentes:

- Número.
- Responsable.
- Período desde.
- Período hasta.
- Motivo.
- Lugar de trabajo.
- Estado.
- Fechas del proceso.
- Observaciones.

3. Formulario de creación

Campos visibles:

- Responsable.
- Período desde.
- Período hasta.
- Motivo.
- Lugar de trabajo.
- Observaciones.

Campos automáticos:

- Número.
- Estado **BORRADOR**.
- Usuario creador.
- Snapshot del responsable.
- Fecha de creación.

4. Reglas de cabecera

1. El responsable debe estar activo.
2. El período hasta no puede ser anterior al período desde.
3. El motivo debe ser obligatorio para presentación, aunque puede admitirse vacío durante el primer guardado del borrador.
4. El número se genera automáticamente.
5. El número no se reutiliza.
6. El estado inicial siempre es **BORRADOR**.
7. Las fechas de presentación, aprobación y cierre permanecen vacías.

8. El usuario solo puede crear rendiciones para sí mismo, salvo permiso administrativo.
 9. El período no debe exceder el máximo definido por política.
 10. Se debe advertir si existe otra rendición del responsable para el mismo período.
 11. Una coincidencia de período puede ser válida; debe advertirse antes de bloquear.
 12. El responsable queda congelado mediante snapshot.
-

5. Estados editables

Editable normalmente

```
BORRADOR  
OBSERVADA
```

Solo consulta

```
PRESENTADA  
EN_REVISION  
APROBADA  
RECHAZADA  
LIQUIDADA  
CERRADA  
ANULADA
```

La edición de una rendición observada debe limitarse a los campos y gastos autorizados por la observación.

6. Servicio

```
services/rendiciones/cabecera.py
```

Funciones:

```
crear_rendicion()  
actualizar_borrador()  
generar_numero_rendicion()  
anular_borrador()  
validar_periodo_rendicion()
```

Ejemplo conceptual:

```
@transaction.atomic  
def crear_rendicion(  
    *,
```



```
        responsable,  
        periodo_desde,  
        periodo_hasta,  
        motivo,  
        lugar_trabajo,  
        observaciones,  
        usuario,  
    ):  
        numero = generar_numero_rendicion()  
  
        return Rendicion.objects.create(  
            numero=numero,  
            responsable=responsable,  
            responsable_nombre=responsable.nombre,  
            responsable_rut=responsable.rut,  
            responsable_cargo=responsable.cargo,  
            periodo_desde=periodo_desde,  
            periodo_hasta=periodo_hasta,  
            motivo=motivo,  
            lugar_trabajo=lugar_trabajo,  
            observaciones=observaciones,  
            estado=Rendicion.Estado.BORRADOR,  
            creado_por=usuario,  
        )
```

7. URLs

```
/administracion/rendiciones/nueva/  
/administracion/rendiciones/<pk>/  
/administracion/rendiciones/<pk>/editar/  
/administracion/rendiciones/<pk>/anular-borrador/
```

8. Vistas

```
RendicionCreateView  
RendicionDetailView  
RendicionUpdateView  
RendicionAnularBorradorView
```

9. Pantalla de edición

Secciones:

Cabecera
Fondos recibidos
Gastos
Documentos
Validaciones
Resumen

Durante R002 solo se implementará inicialmente la cabecera; las demás secciones se incorporarán en R005–R009.

10. Permisos

view_rendicion
add_rendicion
change_rendicion
crear_rendicion ajena
anular_rendicion_borrador

Reglas por objeto:

puede_ver_rendicion(usuario, rendicion)
puede_editar_rendicion(usuario, rendicion)
puede_anular_borrador(usuario, rendicion)

11. Auditoría

Registrar:

- Creación.
- Cambio de período.
- Cambio de motivo.
- Cambio de lugar.
- Cambio de responsable, cuando esté excepcionalmente permitido.
- Anulación.

12. Pruebas

- Crear borrador.
- Número automático.
- Período válido.
- Período inválido.
- Responsable inactivo.

- Crear para sí mismo.
 - Crear para otro sin permiso.
 - Editar borrador.
 - Editar observada.
 - Impedir editar presentada.
 - Snapshot del responsable.
 - Dos creaciones concurrentes.
-

13. Criterios de aceptación

- Se crea una rendición en borrador.
 - La numeración es única.
 - El período se valida.
 - El responsable queda históricamente identificado.
 - Solo los estados autorizados son editables.
 - Los permisos se verifican en backend.
 - Las modificaciones quedan auditadas.
-

R001 — Lista y búsqueda de rendiciones

1. Objetivo

Permitir localizar rendiciones mediante filtros y mostrar su situación operativa y financiera.

2. URL

```
/administracion/rendiciones/
```

Nombre:

```
administracion:rendicion_list
```

3. Vista

```
class RendicionListView(  
    LoginRequiredMixin,  
    PermissionRequiredMixin,  
    ListView,  
):  
    model = Rendicion
```

```
permission_required = "administracion.view_rendicion"
paginate_by = 50
```

La consulta se construirá en:

```
selectors/rendiciones.py
```

4. Alcance según usuario

Responsable

Ve:

- Sus propias rendiciones.
- Las rendiciones donde tenga delegación explícita.

Revisor

Ve:

- Rendiciones presentadas o en revisión dentro de su ámbito.

Finanzas

Ve:

- Rendiciones aprobadas.
- Liquidaciones.
- Pendientes de devolución o reembolso.

Administrador o auditor

Ve todas, según permisos.

5. Filtros

Identificación

- Número.
- Responsable.
- RUT.
- Área.
- Cargo.

Estado

- Borrador.

- Presentada.
- En revisión.
- Observada.
- Aprobada.
- Rechazada.
- Liquidada.
- Cerrada.
- Anulada.

Fechas

- Período desde/hasta.
- Fecha de creación.
- Fecha de presentación.
- Fecha de aprobación.
- Fecha de cierre.

Situación financiera

- Con fondos.
- Sin fondos.
- Con diferencia a devolver.
- Con diferencia a reembolsar.
- Cuadrada.
- Con gastos sin documento.
- Con observaciones.
- Con validaciones pendientes.

Texto

- Número.
- Motivo.
- Lugar.
- Observaciones.
- Responsable.

6. QuerySet

Se deben anotar:

```
total_fondos_db
total_declarado_db
total_aprobado_db
cantidad_detalle
cantidad_observados
cantidad_sin_documento
cantidad_errores_documentales
diferencia_preliminar_db
```

La vista no debe ejecutar consultas por cada fila.

7. Columnas

Columna	Contenido
Número	Identificador
Responsable	Snapshot histórico
Período	Desde/hasta
Fondos	Fondos activos
Rendido	Gastos declarados
Aprobado	Gastos aprobados
Diferencia	Preliminar o definitiva
Estado	Badge
Validaciones	Errores o advertencias
Acciones	Ver, editar, presentar

8. Resumen del filtro

Mostrar:

- Cantidad de rendiciones.
 - Total de fondos.
 - Total rendido.
 - Total aprobado.
 - Total pendiente de devolución.
 - Total pendiente de reembolso.
-

9. Ordenamiento

- Más recientes.
- Número.
- Responsable.
- Período.
- Estado.
- Mayor fondo.
- Mayor diferencia.

Por defecto:

```
período hasta descendente  
id descendente
```

10. Acciones

Según estado y permiso:

- Ver.
- Editar.
- Adjuntar.
- Presentar.
- Revisar.
- Liquidar.
- Exportar.

En este bloque solo quedarán habilitadas las acciones ya implementadas.

11. Templates

```
templates/administracion/rendiciones/  
├─ list.html  
├─ _filtros.html  
├─ _tabla.html  
└─ _resumen.html
```

12. Pruebas

- Responsable ve solo las propias.
 - Revisor ve pendientes.
 - Administrador ve todas.
 - Filtro por número.
 - Filtro por responsable.
 - Filtro por estado.
 - Filtro por período.
 - Filtro por diferencia.
 - Filtro sin documento.
 - Paginación.
 - Totales.
 - Ausencia de N+1.
 - Permisos.
-

13. Criterios de aceptación

- Las rendiciones pueden localizarse.
- Los filtros se combinan.
- La paginación conserva filtros.
- Los totales son correctos.
- El alcance depende del usuario.
- La consulta es eficiente.
- Las acciones dependen del estado.

R005 — Entregas de fondo

1. Objetivo

Registrar uno o varios fondos entregados a un responsable para financiar gastos posteriormente rendidos.

Ejemplo:

Entrega 1: transferencia	\$300.000 CLP
Entrega 2: efectivo	\$100.000 CLP
Total de fondos:	\$400.000 CLP

2. Modelo actual

EntregaFondo

El modelo exige actualmente una **Rendicion**, registra fecha, monto, forma, referencia, anotación y anulación.

3. Problema funcional detectado

En la operación real, el fondo puede entregarse **antes de que exista la rendición**.

Con el modelo actual:

EntregaFondo → Rendicion obligatoria

no puede registrarse un fondo previo.

4. Ajuste recomendado para el MVP

Modificar el modelo para incorporar:

```
responsable = models.ForeignKey(
    ResponsableRendicion,
    on_delete=models.PROTECT,
    related_name="entregas_fondo",
)

rendicion = models.ForeignKey(
    Rendicion,
    on_delete=models.PROTECT,
    null=True,
    blank=True,
    related_name="entregas_fondo",
)
```

Así una entrega puede existir:

```
Fondo entregado
→ pendiente de asociar
→ vinculado posteriormente a una rendición
```

5. Restricción de vinculación

Cuando se vincule a una rendición:

```
EntregaFondo.responsable
=
Rendicion.responsable
```

No se permitirá vincular el fondo de una persona a la rendición de otra.

6. Posible evolución

Si una entrega debe repartirse entre varias rendiciones, será necesario crear:

```
class AplicacionFondoRendicion(ModeloAuditoria):
    entrega_fondo = models.ForeignKey(...)
    rendicion = models.ForeignKey(...)
    monto = models.DecimalField(...)
```

Para el MVP puede mantenerse:

Una entrega → una rendición
Una rendición → varias entregas

7. Formulario

Campos:

- Responsable.
- Fecha.
- Monto.
- Forma de entrega.
- Referencia.
- Rendición, opcional.
- Anotación.
- Comprobante.

8. Formas de entrega

Actualmente:

- Transferencia.
- Efectivo.
- Giro de cajero.
- Débito.
- Otro.

Podría agregarse:

TARJETA_EMPRESA

solo si la tarjeta corporativa se considerará un fondo entregado y no un mecanismo de pago separado.

9. Reglas de negocio

1. El monto debe ser positivo.
2. El responsable debe estar activo al registrar.
3. La fecha es obligatoria.
4. La referencia será obligatoria para transferencias.
5. Una entrega vinculada debe corresponder al mismo responsable.
6. Una entrega anulada no suma fondos.
7. Una entrega conciliada con banco no se edita libremente.
8. No se elimina físicamente una entrega.

9. Anular exige motivo.
10. El fondo puede registrarse antes de la rendición.
11. Una rendición cerrada no admite nuevos fondos.
12. La moneda debe coincidir con la rendición y cuenta bancaria.

10. Estado de la entrega

Se recomienda agregar:

```
class Estado(models.TextChoices):
    PENDIENTE_VINCULAR = (
        "PENDIENTE_VINCULAR",
        "Pendiente de vincular",
    )
    VINCULADA = "VINCULADA", "Vinculada"
    CONCILIADA = "CONCILIADA", "Conciliada"
    ANULADA = "ANULADA", "Anulada"
```

El booleano `anulada` puede migrarse progresivamente al estado.

11. URLs

```
/administracion/rendiciones/fondos/
/administracion/rendiciones/fondos/nuevo/
/administracion/rendiciones/fondos/<pk>/
/administracion/rendiciones/fondos/<pk>/editar/
/administracion/rendiciones/fondos/<pk>/vincular/
/administracion/rendiciones/fondos/<pk>/anular/
/administracion/rendiciones/<pk>/fondos/nuevo/
```

12. Vistas

```
EntregaFondoListView
EntregaFondoDetailView
EntregaFondoCreateView
EntregaFondoUpdateView
EntregaFondoVincularView
EntregaFondoAnularView
```

13. Servicio

```
services/rendiciones/fondos.py
```

Funciones:

```
registrar_entrega_fondo()  
vincular_fondo_rendicion()  
modificar_entrega_fondo()  
anular_entrega_fondo()  
calcular_fondos_pendientes_responsable()
```

14. Vinculación bancaria

Posteriormente, B014 y R017 utilizarán:

```
MovimientoBancario de egreso  
→ AplicacionMovimientoBancario  
→ EntregaFondo
```

La entrega no debe considerarse conciliada solo por estar registrada.

15. Pruebas

- Fondo previo a rendición.
- Fondo vinculado al crear.
- Múltiples fondos.
- Responsable diferente.
- Referencia de transferencia.
- Monto cero.
- Monto negativo.
- Anulación.
- Fondo conciliado.
- Rendición cerrada.
- Total de fondos.
- Permisos.

16. Criterios de aceptación

- Se registran múltiples entregas.
- Pueden existir antes de la rendición.
- Se vinculan al responsable correcto.

- Las anuladas no suman.
- El total acumulado es correcto.
- Los fondos históricos no se eliminan.
- La relación bancaria queda preparada.

R006 — Detalle de gastos

1. Objetivo

Registrar cada gasto individual que compone una rendición.

Tipos iniciales:

Factura
Boleta
Peaje
Recibo
Otro
Sin documento

Estos tipos ya existen en `DetalleRendicion`.

2. Modelo principal

DetalleRendicion

Campos existentes:

- Rendición.
- Fecha.
- Tipo de documento.
- Número.
- Proveedor.
- RUT.
- Sucursal.
- Categoría.
- Subcategoría.
- Descripción.
- Forma de pago.
- Neto.
- IVA.
- Total.
- Estado de revisión.
- Monto aprobado.
- Observación.

- Justificación.
 - Comprobante.
-

3. Campos adicionales recomendados

```
moneda = models.CharField(  
    max_length=3,  
    default="CLP",  
)  
  
tipo_cambio = models.DecimalField(  
    max_digits=14,  
    decimal_places=6,  
    null=True,  
    blank=True,  
)  
  
total_clp = models.DecimalField(  
    max_digits=14,  
    decimal_places=2,  
    null=True,  
    blank=True,  
)  
  
fingerprint = models.CharField(  
    max_length=64,  
    blank=True,  
)
```

Los campos de moneda extranjera solo deben habilitarse si existen gastos reales fuera de CLP.

También se recomienda incorporar los campos de anulación definidos anteriormente.

4. Formas de pago

El catálogo actual contiene:

- Efectivo.
- Transferencia.
- Débito.
- Otro.

Se recomienda evaluar:

CRÉDITO
TARJETA_EMPRESA

porque tienen implicaciones distintas:

- Tarjeta personal: puede requerir reembolso.
- Tarjeta de empresa: no representa dinero entregado al responsable.
- Efectivo proveniente de fondo: consume fondo entregado.

5. Reglas de documento

Factura

Requiere normalmente:

- Número.
- Proveedor.
- RUT.
- Neto.
- IVA.
- Total.
- Adjunto.

Boleta

Requiere:

- Proveedor o comercio.
- Total.
- Fecha.
- Adjunto.

El número puede no estar siempre disponible o ser poco confiable.

Peaje

Requiere:

- Fecha.
- Total.
- Ruta, plaza o descripción.
- Comprobante cuando exista.

Sin documento

Requiere:

- Justificación.
- Categoría que permita gastos sin documento.

- Monto dentro de política.
- Posterior revisión especial.

6. Validación de montos

Cuando se informen neto e IVA:

$$\text{total} = \text{neto} + \text{IVA}$$

Se admitirá una tolerancia mínima de redondeo configurada.

No se debe modificar automáticamente el total sin informar al usuario si existe una diferencia.

7. Validación de categoría

1. Categoría activa.
2. Subcategoría activa.
3. La subcategoría pertenece a la categoría.
4. Las reglas documentales de la categoría se aplican.
5. Una rendición presentada no admite categorías nuevas sin reapertura.

8. Validación de fecha

La fecha debería estar dentro del período de la rendición.

Política recomendada:

- Dentro del período: válido.
- Fuera por pocos días: advertencia justificable.
- Fuera del límite permitido: error bloqueante.

La tolerancia debe definirse en configuración.

9. URLs

```
/administracion/rendiciones/<rendicion_id>/gastos/  
/administracion/rendiciones/<rendicion_id>/gastos/nuevo/  
/administracion/rendiciones/gastos/<pk>/  
/administracion/rendiciones/gastos/<pk>/editar/  
/administracion/rendiciones/gastos/<pk>/anular/
```


10. Vistas

```
DetalleRendicionListView  
DetalleRendicionCreateView  
DetalleRendicionDetailView  
DetalleRendicionUpdateView  
DetalleRendicionAnularView
```

11. Formulario

Secciones:

Documento

- Tipo.
- Número.
- Fecha.

Proveedor

- Nombre.
- RUT.
- Sucursal.

Clasificación

- Categoría.
- Subcategoría.
- Descripción.

Pago

- Forma de pago.
- Moneda.
- Neto.
- IVA.
- Total.

Respaldo

- Adjuntos.
- Justificación.

12. Servicio

```
services/rendiciones/gastos.py
```

Funciones:

```
registrar_gasto()  
modificar_gasto()  
anular_gasto()  
calcular_fingerprint_gasto()  
validar_montos_gasto()  
validar_periodo_gasto()
```

13. Permisos

```
view_detallerendicion  
add_detallerendicion  
change_detallerendicion  
anular_detallerendicion
```

Se aplicarán reglas por objeto según:

- Responsable.
- Estado de rendición.
- Rol administrativo.

14. Auditoría

Registrar:

- Alta.
- Cambios de documento.
- Cambio de monto.
- Cambio de categoría.
- Cambio de proveedor.
- Anulación.
- Motivo.

15. Pruebas

- Factura válida.
- Boleta válida.
- Peaje.
- Gasto sin documento.
- Justificación obligatoria.
- Total incorrecto.
- Categoría inactiva.

- Subcategoría incorrecta.
 - Fecha fuera del período.
 - Gasto duplicado.
 - Rendición presentada.
 - Anulación.
 - Precisión decimal.
-

16. Criterios de aceptación

- Se registran los tipos de gasto definidos.
 - Los montos son consistentes.
 - Categoría y subcategoría coinciden.
 - Los gastos sin respaldo se justifican.
 - Los gastos duplicados se advierten.
 - Solo se editan en estados autorizados.
 - Las modificaciones quedan auditadas.
-

R007 — Adjuntos y previsualización

1. Objetivo

Permitir agregar múltiples documentos a cada gasto y visualizarlos de manera segura.

La Carta Gantt requiere:

- Subida.
 - Vista previa.
 - Descarga.
 - Reemplazo controlado.
-

2. Arquitectura

Se reutilizará el modelo transversal definido en A006:

AdjuntoAdministracion

Relación:

DetalleRendicion
→ múltiples AdjuntoAdministracion

El campo actual:

DetalleRendicion.comprobante

se conservará temporalmente para migrar archivos existentes.

3. Categorías de adjunto

COMPROBANTE_PRINCIPAL
COMPROBANTE_ADICIONAL
FACTURA
BOLETA
PEAJE
RECIBO
RESPALDO_TRANSFERENCIA
JUSTIFICACION
OTRO

4. Reglas de archivo

Formatos iniciales:

PDF
JPG
JPEG
PNG

Opcionalmente:

HEIC

solo si se incorpora una conversión segura para previsualización.

No se deben admitir ejecutables ni formatos con macros.

5. Previsualización

Imágenes

- Miniatura.
- Vista ampliada.
- Rotación visual.
- Zoom.

PDF

- Primera página o visor interno.
- Descarga protegida.

Otros formatos autorizados

- Icono.
 - Metadatos.
 - Descarga.
-

6. Reemplazo controlado

No se sobrescribirá físicamente el archivo anterior.

Flujo:

Adjunto anterior

- se marca inactivo
- se registra motivo
- se carga nuevo adjunto
- se conserva historial

7. Metadatos

Mostrar:

- Nombre original.
 - Tipo.
 - Tamaño.
 - Hash.
 - Fecha de carga.
 - Usuario.
 - Estado.
 - Categoría.
-

8. Seguridad

1. El archivo no se sirve directamente por URL pública.
2. La descarga verifica permiso sobre la rendición.
3. Se evita path traversal.
4. Se valida MIME y extensión.
5. Se calcula SHA-256.
6. Se registra descarga sensible.
7. Los nombres físicos no utilizan el nombre proporcionado directamente.

8. Los archivos inactivos solo son visibles para auditoría.

9. Control de almacenamiento

Dado que los respaldos pueden aumentar considerablemente el volumen de `media`, se recomienda guardar:

- Tamaño de archivo.
- Hash.
- Fecha.
- Tipo.
- Estado.

Esto permitirá posteriormente crear:

- Indicador de uso de almacenamiento.
 - Reportes por módulo.
 - Política de respaldos.
 - Detección de archivos duplicados.
 - Archivado de documentos antiguos.
-

10. URLs

```
/administracion/rendiciones/gastos/<pk>/adjuntos/  
/administracion/rendiciones/gastos/<pk>/adjuntos/subir/  
/administracion/adjuntos/<uuid>/ver/  
/administracion/adjuntos/<uuid>/descargar/  
/administracion/adjuntos/<uuid>/reemplazar/  
/administracion/adjuntos/<uuid>/desactivar/
```

11. Componente visual

```
templates/administracion/componentes/adjuntos.html
```

Cada gasto mostrará:

```
[Vista previa] [Descargar] [Reemplazar] [Desactivar]
```

según permisos y estado.

12. Servicio

```
services/rendiciones/adjuntos.py
```

Funciones:

```
subir_adjunto_gasto()  
reemplazar_adjunto_gasto()  
desactivar_adjunto_gasto()  
obtener_adjunto_protegido()  
generar_miniatra()
```

13. Pruebas

- Subir imagen.
- Subir PDF.
- Múltiples archivos.
- Extensión inválida.
- MIME incorrecto.
- Archivo grande.
- Archivo vacío.
- Vista previa.
- Descarga autorizada.
- Descarga denegada.
- Reemplazo.
- Historial.
- Hash duplicado.

14. Criterios de aceptación

- Un gasto admite múltiples adjuntos.
 - Las imágenes y PDF se previsualizan.
 - La descarga es protegida.
 - Los reemplazos no destruyen historial.
 - Se conservan metadatos.
 - Se registra el tamaño.
 - Los archivos existentes pueden migrarse.
-

R008 — Validación documental

1. Objetivo

Ejecutar controles automáticos sobre cada gasto antes de que la rendición sea presentada o revisada.

La Carta Gantt exige controlar:

- Duplicados.
- Montos.
- Fechas.
- Proveedor.
- Falta de respaldo.

2. Modelo propuesto

```
class ValidacionDetalleRendicion(ModeloAuditoria):
    class Severidad(models.TextChoices):
        ERROR = "ERROR", "Error"
        ADVERTENCIA = "ADVERTENCIA", "Advertencia"
        INFORMACION = "INFORMACION", "Información"

    class Estado(models.TextChoices):
        ABIERTA = "ABIERTA", "Abierta"
        RESUELTA = "RESUELTA", "Resuelta"
        JUSTIFICADA = "JUSTIFICADA", "Justificada"

    detalle = models.ForeignKey(
        DetalleRendicion,
        on_delete=models.CASCADE,
        related_name="validaciones",
    )
    codigo = models.CharField(max_length=60)
    severidad = models.CharField(
        max_length=20,
        choices=Severidad.choices,
    )
    estado = models.CharField(
        max_length=20,
        choices=Estado.choices,
        default=Estado.ABIERTA,
    )
    campo = models.CharField(max_length=100, blank=True)
    mensaje = models.TextField()
    datos = models.JSONField(default=dict, blank=True)
    justificacion = models.TextField(blank=True)
    resuelta_en = models.DateTimeField(null=True, blank=True)
```



```

    resuelta_por = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="validaciones_rendicion_resueltas",
    )

```

3. Códigos iniciales

```

DOCUMENTO_DUPLICADO
POSIBLE_DUPLICADO
MONTO_INCONSISTENTE
TOTAL_NO_COINCIDE
FECHA_FUERA_PERIODO
FECHA_FUTURA
PROVEEDOR_FALTANTE
RUT_PROVEEDOR_INVALIDO
NUMERO_DOCUMENTO_FALTANTE
CATEGORIA_FALTANTE
SUBCATEGORIA_INVALIDA
ADJUNTO_FALTANTE
GASTO_SIN_DOCUMENTO
JUSTIFICACION_FALTANTE
MONTO_SIN_DOCUMENTO_EXCEDIDO
FORMA_PAGO_INCONSISTENTE

```

4. Fingerprint documental

Base recomendada:

```

tipo_documento
+ RUT proveedor normalizado
+ número documento normalizado
+ fecha
+ total

```

Para documentos sin número:

```

proveedor
+ fecha
+ total
+ descripción normalizada

```

El fingerprint se utiliza como señal, no como única decisión.

5. Duplicado exacto

Un documento debe considerarse duplicado exacto cuando coincidan:

- Tipo.
- RUT.
- Número.
- Total.

Especialmente para facturas.

El sistema debe buscar en:

- La misma rendición.
 - Otras rendiciones del responsable.
 - Rendiciones de otros responsables.
 - Gastos anulados, como advertencia histórica.
-

6. Posible duplicado

Puede generarse una advertencia cuando coincidan:

- Fecha.
- Proveedor.
- Total.

pero cambie el número.

Esto no debe bloquear automáticamente, porque pueden existir gastos legítimos iguales.

7. Reglas por severidad

Error bloqueante

Impide presentar:

- Total no positivo.
- Documento exacto duplicado no justificado.
- Categoría ausente.
- Subcategoría incoherente.
- Justificación obligatoria ausente.
- Documento obligatorio ausente.
- Total inconsistente.

Advertencia

Permite guardar, pero requiere revisión o justificación:

- Fecha fuera del período permitido.
 - Posible duplicado.
 - Proveedor sin RUT.
 - Número de boleta ausente.
 - Gasto sin respaldo autorizado.
 - Forma de pago poco clara.
-

8. Ejecución

La validación se ejecutará:

- Al guardar un gasto.
 - Al modificarlo.
 - Al cargar o desactivar adjuntos.
 - Antes de presentar la rendición.
 - Por solicitud manual del usuario.
-

9. Servicio

```
services/rendiciones/validacion_documental.py
```

Funciones:

```
validar_detalle()  
validar_rendicion_completa()  
detectar_documento_duplicado()  
validar_montos()  
validar_fecha()  
validar_proveedor()  
validar_adjuntos()  
resolver_validaciones_obsoletas()
```

10. Interfaz

Cada gasto mostrará:

✓ Válido
△ 2 advertencias
× 1 error

Al abrir:

Severidad	Control	Mensaje	Estado	Acción
-----------	---------	---------	--------	--------

11. Justificación

Una advertencia podrá justificarse.

Un error solo podrá justificarse cuando la política lo permita expresamente.

La justificación debe registrar:

- Usuario.
- Fecha.
- Motivo.
- Validación.
- Estado anterior y nuevo.

12. Permisos

```
view_validaciondetallerendicion  
justificar_validacionrendicion  
resolver_validacionrendicion
```

El responsable podrá justificar advertencias propias.

Solo un revisor o administrador podrá resolver determinados errores.

13. Pruebas

- Factura duplicada.
- Posible duplicado.
- Documento en otra rendición.
- Total inconsistente.
- Fecha fuera del período.
- Fecha futura.
- Proveedor vacío.
- RUT inválido.

- Adjunto faltante.
 - Sin documento con justificación.
 - Sin documento sin justificación.
 - Reejecución de validación.
 - Resolución automática.
 - Justificación autorizada.
 - Justificación no autorizada.
-

14. Criterios de aceptación

- Cada gasto puede validarse.
 - Los problemas quedan estructurados.
 - Se distinguen errores y advertencias.
 - Los duplicados se detectan.
 - Las validaciones se reevalúan después de corregir.
 - Las justificaciones quedan auditadas.
 - Los errores bloqueantes impiden presentación.
-

R009 — Cálculos de rendición

1. Objetivo

Calcular de forma consistente los fondos recibidos, los gastos declarados, los montos aprobados y las diferencias.

La Carta Gantt exige que totales, fondos, aprobaciones y saldos sean consistentes.

2. Totales requeridos

2.1 Total de fondos

Suma de entregas no anuladas vinculadas

El modelo actual ya calcula `total_fondos`.

2.2 Total declarado

Suma de todos los gastos no anulados

No debe excluirse un gasto solo porque todavía no fue revisado.

Se recomienda renombrar conceptualmente:

```
total_rendido  
→ total_declarado
```

2.3 Total observado

```
Suma de gastos en estado OBSERVADO
```

2.4 Total rechazado

```
Suma de gastos en estado RECHAZADO
```

2.5 Total aprobado

```
Suma de monto_aprobado  
de gastos APROBADOS
```

El modelo actual ya implementa una primera versión de este cálculo.

2.6 Total sin documento

```
Suma de gastos con tipo SIN_DOCUMENTO
```

2.7 Diferencia preliminar

Mientras la rendición todavía no está aprobada:

```
diferencia preliminar  
= total fondos  
- total declarado
```

Resultado positivo:

El responsable conserva fondos sin rendir

Resultado negativo:

El responsable gastó más que los fondos entregados

2.8 Diferencia aprobada

Después de revisión:

diferencia aprobada
= total fondos
- total aprobado

Resultado igual a cero

Rendición cuadrada

Resultado positivo

Responsable debe devolver

Resultado negativo

Empresa debe reembolsar

3. Propiedades recomendadas

```
@property
def total_declarado(self):
    total = self.detalles.filter(
        anulado=False
    ).aggregate(
        total=Sum("total")
    )["total"]

    return total or Decimal("0.00")
```

```
@property
def diferencia_preliminar(self):
    return self.total_fondos - self.total_declarado
```

```
@property
def diferencia_aprobada(self):
    return self.total_fondos - self.total_aprobado
```

```
@property
def monto_a_devolver(self):
    return max(
        self.diferencia_aprobada,
        Decimal("0.00"),
    )
```

```
@property
def monto_a_reembolsar(self):
    return max(
        -self.diferencia_aprobada,
        Decimal("0.00"),
    )
```

4. Momento de cálculo

Los totales deben recalcularse al:

- Agregar gasto.
- Editar gasto.
- Anular gasto.
- Cambiar estado de revisión.
- Cambiar monto aprobado.
- Registrar fondo.
- Anular fondo.
- Vincular fondo.
- Presentar.
- Aprobar.
- Liquidar.

5. Persistir o calcular

Durante borrador

Se recomienda calcular dinámicamente mediante agregaciones.

Durante aprobación y liquidación

Se debe guardar un snapshot en `LiquidacionRendicion` para que el resultado histórico no cambie.

Esto permite distinguir:

Cálculo actual operativo
vs.
Cálculo histórico aprobado

6. Selector optimizado

`selectors/calculos_rendiciones.py`

Funciones:

`anotar_totales_rendiciones()`
`obtener_resumen_rendicion()`
`calcular_diferencia_preliminar()`
`calcular_diferencia_aprobada()`

El listado R001 debe usar anotaciones y no llamar propiedades que generen consultas individuales.

7. Pantalla resumen

En el detalle:

Concepto	Monto
Fondos recibidos	\$400.000 CLP
Gastos declarados	\$450.000 CLP
Gastos observados	\$20.000 CLP
Gastos rechazados	\$10.000 CLP
Monto aprobado	\$420.000 CLP
Diferencia preliminar	-\$50.000 CLP
Diferencia aprobada	-\$20.000 CLP
Empresa debe reembolsar	\$20.000 CLP

Antes de aprobar se destacará que la diferencia es preliminar.

8. Reglas de negocio

1. Todos los cálculos usan `Decimal`.
 2. Los gastos anulados no suman.
 3. Los fondos anulados no suman.
 4. Los gastos rechazados no forman parte del aprobado.
 5. Los gastos observados permanecen en el declarado.
 6. El monto aprobado no supera el gasto.
 7. No se redondean totales con `float`.
 8. Las monedas deben ser compatibles.
 9. Una liquidación usa snapshot.
 10. Los totales del listado y detalle deben coincidir.
 11. Los cambios deben actualizar la interfaz.
 12. Las diferencias no se ingresan manualmente.
-

9. Pruebas

- Sin fondos ni gastos.
 - Un fondo.
 - Múltiples fondos.
 - Fondo anulado.
 - Un gasto.
 - Múltiples gastos.
 - Gasto anulado.
 - Gasto observado.
 - Gasto rechazado.
 - Aprobación parcial.
 - Diferencia cero.
 - Responsable devuelve.
 - Empresa reembolsa.
 - Decimales.
 - Totales del listado.
 - Snapshot de liquidación.
-

10. Criterios de aceptación

- Los fondos suman correctamente.
 - Los gastos declarados son correctos.
 - Los montos aprobados son correctos.
 - Se distinguen diferencias preliminares y definitivas.
 - Se determina devolución o reembolso.
 - Los anulados no afectan los cálculos.
 - Los totales coinciden entre vistas.
 - Los cálculos históricos quedan congelados al liquidar.
-

7. Estructura de archivos del bloque

```
administracion/
├── models/
│   ├── rendiciones.py
│   ├── adjuntos.py
│   └── validaciones.py
├── forms/
│   ├── responsables.py
│   ├── categorias_gasto.py
│   ├── rendiciones.py
│   ├── entregas_fondo.py
│   ├── detalles_rendicion.py
│   └── validaciones_rendicion.py
├── views/
│   ├── responsables.py
│   ├── categorias_gasto.py
│   ├── rendiciones.py
│   ├── entregas_fondo.py
│   ├── detalles_rendicion.py
│   ├── adjuntos_rendicion.py
│   └── validaciones_rendicion.py
├── services/
│   └── rendiciones/
│       ├── cabecera.py
│       ├── responsables.py
│       ├── categorias.py
│       ├── fondos.py
│       ├── gastos.py
│       ├── adjuntos.py
│       ├── validacion_documental.py
│       └── calculos.py
├── selectors/
│   ├── rendiciones.py
│   ├── fondos_rendicion.py
│   ├── gastos_rendicion.py
│   └── calculos_rendiciones.py
├── templates/administracion/rendiciones/
│   ├── list.html
│   ├── detail.html
│   ├── form.html
│   ├── responsables/
│   └── categorias/
```

```
|   ├── fondos/
|   ├── gastos/
|   ├── adjuntos/
|   ├── validaciones/
|   └── componentes/
|
|   ├── static/administracion/js/
|   │   ├── rendiciones.js
|   │   ├── gastos_rendicion.js
|   │   └── adjuntos_rendicion.js
|   └── tests/rendiciones/
|       ├── test_responsables.py
|       ├── test_categorias.py
|       ├── test_cabecera.py
|       ├── test_lista.py
|       ├── test_fondos.py
|       ├── test_gastos.py
|       ├── test_adjuntos.py
|       ├── test_validacion_documental.py
|       └── test_calculos.py
```

8. Migraciones previstas

Migración 1 — Responsable

Agregar, si se aprueba:

```
correo
teléfono
área
```

Agregar restricciones e índices de RUT.

Migración 2 — Rendición

Agregar:

```
snapshot del responsable
secuencia o mecanismo de numeración
índices por responsable, estado y período
```

Migración 3 — Categorías

Agregar:

código
descripción
reglas documentales

Migración 4 — Entregas

Agregar:

responsable
rendición opcional
estado
datos de anulación

Crear una migración de datos para completar el responsable desde la rendición existente.

Migración 5 — Gastos

Agregar:

anulación
fingerprint
moneda, si corresponde
tipo de cambio, si corresponde

Migración 6 — Validaciones

Crear:

ValidacionDetalleRendicion

Migración 7 — Adjuntos

Migrar progresivamente:

```
DetalleRendicion.comprobante  
→ AdjuntoAdministracion
```

El campo antiguo no se elimina hasta comprobar la migración.

9. Índices recomendados

Rendicion

```
models.Index(fields=["responsable", "estado"])  
models.Index(fields=["periodo_desde", "periodo_hasta"])  
models.Index(fields=["estado", "periodo_hasta"])
```

EntregaFondo

```
models.Index(fields=["responsable", "estado"])  
models.Index(fields=["rendicion", "estado"])  
models.Index(fields=["fecha"])
```

DetalleRendicion

```
models.Index(fields=["rendicion", "estado_revision"])  
models.Index(fields=["fecha"])  
models.Index(fields=["rut_proveedor", "numero_documento"])  
models.Index(fields=["fingerprint"])  
models.Index(fields=["categoria", "subcategoria"])
```

ValidacionDetalleRendicion

```
models.Index(fields=["detalle", "estado"])  
models.Index(fields=["codigo", "estado"])  
models.Index(fields=["severidad", "estado"])
```

10. Definición de terminado del bloque

Este bloque se considerará terminado cuando:

- Se pueden registrar responsables.
- Los responsables pueden vincularse con usuarios.

- Se conservan datos históricos.
- Se administran categorías y subcategorías.
- No existen duplicados funcionales.
- Se crea una rendición en borrador.
- La numeración es automática y segura.
- La lista permite filtros y paginación.
- Se registran fondos antes o después de crear la rendición.
- Se pueden asociar múltiples fondos.
- Se registran gastos estructurados.
- Se validan fechas, proveedores y montos.
- Un gasto admite múltiples adjuntos.
- Los documentos se visualizan y descargan de forma segura.
- Los reemplazos conservan historial.
- Se detectan duplicados documentales.
- Se identifican gastos sin respaldo.
- Se distinguen errores y advertencias.
- Se calculan fondos, declarado y aprobado.
- Se determina preliminarmente devolución o reembolso.
- Los cálculos del listado y detalle coinciden.
- Las operaciones quedan auditadas.
- Las pruebas automáticas pasan.

11. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Modificar datos históricos del responsable	Alto	Snapshot en rendición
Fondo entregado antes de la rendición	Alto	Rendición opcional y responsable directo
Número duplicado	Alto	Secuencia transaccional
Categoría y subcategoría incoherentes	Medio/ alto	Validación y selector dependiente
Documento duplicado	Alto	Fingerprint y validación
Perder comprobantes	Alto	Adjuntos versionados
URL pública de archivo	Alto	Descarga protegida
Gasto sin documento no controlado	Alto	Justificación y validación
Confundir validación con aprobación	Alto	Modelos separados
Total declarado mal calculado	Alto	Excluir solo anulados
Totales lentos en listados	Medio	Anotaciones
Eliminación sin historial	Alto	Anulación lógica

Riesgo	Impacto	Tratamiento
Diferencia histórica cambiante	Alto	Snapshot de liquidación
Crecimiento de almacenamiento	Medio/ alto	Metadatos, hash y reportes de uso

12. Resultado consolidado

Al terminar este bloque existirá el siguiente flujo:

Responsable

- categoría
- rendición en borrador
- fondos entregados
- gastos registrados
- documentos adjuntos
- validación automática
- cálculo preliminar
- rendición lista para presentar

La salida habilita el siguiente bloque:

R010 – Presentación
R011 – Revisión y observaciones
R012 – Aprobación o rechazo
R013 – Historial de aprobaciones
R014 – Liquidación
R015 – Devolución del responsable
R016 – Reembolso al responsable
R017 – Conciliación de liquidaciones